深度前馈网络：通用逼近、损失函数与激活函数

读前须知

- **前置**：多层感知器与反向传播；知道 Sigmoid 长什么样。
- **这一篇的位置**：深度学习的基本骨架。卷积网络、循环网络、生成模型，都是在它上面换掉某一层或某个连接方式。
- **读完能做什么**：解释通用逼近定理能保证什么、不能保证什么；为不同任务选对损失函数；说清 ReLU 为什么取代了 Sigmoid。

一、从多层感知器到深度前馈网络

1.1 定义与差别

深度前馈网络（Deep Feedforward Network）是具有深度结构的前馈神经网络，可以看成是进化版的多层感知器。

与只有一个或两个隐藏层的浅层网络相比，深度前馈网络具有更多的隐藏层数目，从而具备了更强的特征提取能力。深度前馈网络不考虑输入数据可能具备的任何特定结构，也就是不使用关于数据的先验信息。但特征提取能力增强的代价是运算复杂度的提升。

“前馈”这个词的含义是：信息单向流动，从输入层经隐藏层到输出层，没有回路，也没有跨层的跳跃。同一层内的神经元之间不连接。这与后面的循环网络、卷积网络形成了对照——后者要么有时间上的回环，要么利用空间上的局部结构。

1.2 设计中的五个问题

因而，网络架构的建立、损失函数的选择、输出单元和隐藏单元的设计、训练误差的处理等问题就成为深度前馈网络设计中的一系列核心问题。

这四个问题可以归成四层：**架构**决定网络的形状与容量，**损失函数**决定网络朝什么方向优化，**输出单元与隐藏单元**决定每一层能做什么样的变换，而**训练误差的处理**（正则化、早停等）决定学到的东西能不能推广到新数据。本章按这个顺序逐层展开。

二、架构：链式与全连接

2.1 链式架构

在深度前馈网络的设计中，确定架构是首要考虑的关键问题。架构决定着网络中包含多少基本单元，以及这些基本单元之间如何相互连接。

几乎所有前馈网络采用的都是链式架构，即前一层的输出是后一层的输入。在这样的链式架构中，层的数目和每一层中神经元的数目就是网络的主要变量。

写成公式，链式结构就是一组嵌套的复合：

$$h^{(1)} = f^{(1)}\left(W^{(1)}x + b^{(1)}\right), \quad h^{(2)} = f^{(2)}\left(W^{(2)}h^{(1)} + b^{(2)}\right), \quad \cdots, \quad \hat{y} = f^{(L)}\left(W^{(L)}h^{(L-1)} + b^{(L)}\right)$$

第一层的输入是原始数据 x，此后每一层的输入都是上一层的输出。整张网络从参数角度看就是一个巨大的复合函数，其形状由各层的宽度与总层数共同确定。

2.2 全连接与稀疏连接

在层与层之间，深度架构采用的最常见的方式是**全连接**，意味着相邻层次中的任意神经元都两两相连。

全连接是最简单也最复杂的架构。说它简单，是因为连接方式只有一种选择，不需要考虑保留哪些连接或是放弃哪些连接；说它复杂，是因为这种方式需要确定的参数最大，给网络训练带来了很大的麻烦。目前也有大量神经网络使用了稀疏的连接方式，这需要结合应用场景具体分析。

补充说明：参数量可以精确估算。若第 l−1 层有 n 个神经元、第 l 层有 m 个神经元，全连接层的权重数目为 m×n，偏置数目为 m。两层各 1000 个神经元的全连接层就有约一百万参数；若输入是一张 1000×1000 的图像，把一百万个像素直接接到 1000 个神经元上，权重要 10^9 个。改用 3×3 卷积核、64 个输出通道，参数量只有 3×3×1×64=576 个，而且与输入尺寸无关，两者相差六个数量级。稀疏连接的省参数效果正来源于此，代价是丢掉了全连接能表达的一部分模式。选择哪种，本质上是在容量与可训练性之间做权衡。

> **新手提示：层数与宽度不是同一件事**
>
> 想提升容量时，加宽和加深都能增加参数，但效果不同。加宽让单层能表达更复杂的函数，训练相对容易，但参数量增长是平方级的，而且过宽之后继续加宽收益迅速递减。加深让网络能表达更复杂的组合，参数量增长是线性级的，但训练难度显著上升，需要残差连接、归一化等配套。经验规律是：先加深到出现明显性能提升，再加宽；深度带来的收益通常在中等深度区间最明显，继续加深会进入收益递减甚至退化。判断标准不是“哪个更大”，而是哪个更适合数据的层次结构。

三、通用逼近定理的边界

3.1 定理的内容

介绍多层感知器时曾提到了通用逼近的性质，这个性质的严格形式是**通用逼近定理**。

通用逼近定理的内容是：如果一个前馈网络具有单个隐藏层，这个隐藏层又有足够但是有限数目的神经元，这个神经网络就能够以任意精度逼近任意连续函数。

形式化地说，给定连续函数 f 与任意小的正数 ε，存在一个单隐藏层网络 F，使得对所有定义域内的 x 都有：

$$|F(x) - f(x)| < \varepsilon$$

虽然在这个定理的初始证明中，隐藏神经元的传递函数是具有“挤压”性质的非线性函数，但定理的成立性并不依赖于“挤压”这一具体性质，而是只要求传递函数是非线性的。

补充说明：这条定理有几个版本，条件各不相同。Cybenko 在 1989 年针对 S 型传递函数给出了证明；Hornik、Stinchcombe 与 White 在 1989 年把结论推广到任意有界非常数激活函数；到了 1991 年，Hornik 又进一步指出，只要激活函数**不是多项式**，结论就成立——多项式激活会让网络只能表示固定次数的多项式。所以定理的强形式并不要求“挤压”，只要激活函数具备足够的一般性。反过来说，若激活函数是线性的，整张网络退化为一个线性变换，定理立刻失效——非线性是这条定理的必要前提。

3.2 存在性不等于可构造

通用逼近定理是一个**存在性定理**，它说明需要的神经网络是肯定存在的，却并没有指明具体的构造方法。

所以在给定一个目标函数时，我们可以确定单隐藏层的感知器一定能够将它表示出来，却对隐藏层需要多少神经元毫无把握。这个数目很可能是个天文数字，这会让网络结构在计算机上根本无法实现。即使能够设计出这么复杂的算法，要对它进行训练和泛化也近乎天方夜谭。

> **新手提示：定理保证“能表示”，不保证“能学到”**

> 看到“万能逼近”四个字很容易产生误解，以为只要网络够大什么问题都能解决。但这条定理只说三件事：存在、有限、任意精度。它没有说隐藏层要多少个神经元，没有说梯度下降能不能找到那组参数，也没有说找到之后在新数据上表现如何。一个能被表示但无法被训练的函数，在实际中等于不存在。表达力是必要条件，不是充分条件。

四、深度为什么更好

4.1 单元数量的效率

深度前馈网络的出现克服的正是单隐藏层带来的复杂性问题：使用深度架构的模型既能减少表示目标函数时所需要的单元数量，也能有效降低泛化误差，在一定程度上抑制过拟合的发生。深度架构对复杂函数具有更强的表示能力，这一点已经在数学上得到证明。

这个“数学上得到证明”指的不是万能逼近，而是**表达效率**的结论：存在某些函数族，用深度为 k 的网络表示只需多项式级的参数，而用深度为 k−1 的网络表示则需要指数级的参数。换句话说，深度不是“多堆几层也许更好”，而是在特定函数族上具有严格的效率优势。

4.2 层次化假设

除此之外，深度架构也暗含了这样的假设：待学习的复杂函数可以视为若干简单函数的层次化结合。由于深度学习的思想本就借鉴自神经科学，这个假设也就合情合理了。

这个假设的意义在于，它把“为什么深度有效”从数学层面拉回到了数据层面：真实世界中很多任务本身就是分层的。图像由边缘组成形状、由形状组成物体，语言由音素组成词、由词组成句子。如果任务具有这种结构，那么与之同构的网络在样本效率上就会占优。

五、损失函数

5.1 交叉熵与均方误差

任何机器学习算法都可以看成是对某个预设函数的最优化方法，深度前馈网络也不例外。与其他神经网络一样，深度前馈网络也利用梯度信息进行学习，在处理误差时采用的是反向传播方法，利用反向传播求出梯度后再使用随机梯度下降法寻找损失函数的最小值。但深度网络的非线性特性常常会影响到学习算法的收敛性能，这也是使用非线性函数的神经网络的一个固有特性。

在学习中，损失函数的选择是深度神经网络设计中另一个重要环节。深度前馈网络选择损失函数的准则与其他机器学习算法并无二致：回归问题的损失函数通常是最小均方误差，而分类问题的损失函数通常是**交叉熵**（Cross-Entropy）。

交叉熵描述的是真实的数据分布和神经网络输出分布之间的差异性。当我们使用神经网络的输出去预测符合真实分布的测试数据时，网络输出并不能完全消除关于测试数据的不确定性。这部分剩余的不确定性可以由泛化误差体现，在信息论中则由交叉熵来定义。

如果 p(x) 和 q(x) 是两个离散分布，则两者的交叉熵可以表示为：

$$H(p,q) = -\sum_{x} p(x) \log q(x)$$

在深度前馈网络中，损失函数被定义为分类结果 y 关于训练数据 x 的条件概率的负对数之和：

$$J(\theta) = -\sum_{k} \ln p\left(y_k \mid x_k; \theta\right)$$

确定损失函数后，网络的训练过程就是找到使以上表达式最小的参数 θ 的过程。

补充说明：交叉熵与 KL 散度只差一个常数。把交叉熵拆开：

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q)$$

其中 H(p) 是真实分布自身的熵，与模型参数无关；D_KL 是相对熵，衡量两个分布的差异。所以最小化交叉熵等价于最小化 KL 散度，也就等价于让 q 尽量贴近 p。实践中常用 softmax 的输出作为 q，此时交叉熵对 logits 的梯度恰好是"预测概率减真实标签"，形式非常干净，这也是分类任务几乎一律用交叉熵的原因之一。

> **新手提示：分类不要用均方误差**
>
> 一个初学者常犯的错误是：输出层用了 softmax，损失却用均方误差。这样做能训练，但收敛很慢。原因是 softmax 加均方误差形成的复合函数在预测严重错误时梯度反而很小——预测值与标签差距越大，梯度越接近零，网络学得越慢。而交叉熵对 logits 的梯度是"预测概率减真实标签"，错误越大梯度越大，收敛速度完全不同。回归任务用均方误差、分类任务用交叉熵，这条分工不是习惯问题，而是梯度性质决定的。

5.2 最大似然视角

其实无论是最小均方误差还是交叉熵，体现的都是概率论中**最大似然估计**的原理。最小均方误差与最大似然估计的关系在线性回归中已经建立：假设噪声服从高斯分布，对数似然取负后去掉常数项，得到的正是均方误差。反过来，若强行假设分类的条件概率服从高斯分布，交叉熵的最小化与最小均方误差也会殊途同归——这说明两者的分岔点不在优化方法，而在对数据分布的假设。

这个统一的视角很有价值：它意味着损失函数不是随手挑的，而是对"数据是怎样生成的"这一假设的直接翻译。假设噪声服从高斯分布，最大似然给出均方误差；假设标签服从伯努利分布，最大似然给出二元交叉熵；假设标签服从多项分布，最大似然给出多元交叉熵。换损失函数，本质上是在换对数据的假设。

六、输出单元

6.1 线性单元

使用对数似然作为损失函数的一个优点是它的通用性，概率论上的直观意义使它适用于几乎所有的模型。另一个好处则体现在它可以避免因损失函数饱和造成的梯度消失，这是由于对数操作可以抵消掉大部分传递函数中的指数效果，从而更清晰地展示出变化趋势。

损失函数的表示与输出单元的选择密切相关，输出单元的传递函数决定了交叉熵的具体表达式。输出层的作用是对隐藏层提取出的特征施加额外的变换以得到输出，变换的形式则有多种选择。

最简单的变换形式就是线性变换，它将隐藏特征的线性组合作为输出，简单而实用。回归任务通常就用线性输出：网络主体负责把输入映射到一个特征空间，最后一层做一次线性读出，得到连续值。

6.2 Sigmoid 与 Softmax

除了线性变换，对数几率函数也是一种常用的变换函数，其形式在逻辑回归中已经做过介绍。对数几率函数得到的是一类软输出，因为它能将隐藏特征的线性组合映射到概率之上：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

将对数几率函数加以推广就可以得到 **softmax** 函数，也叫柔性最大值函数。对数几率函数只能解决二分类问题，softmax 函数则可以解决多分类问题。softmax 的形式同样在逻辑回归中做了介绍：

$$\mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

这里有一个需要澄清的常见误解：**softmax 处理的是互斥的多分类**，也就是每个样本只属于一个类别，输出向量各分量之和为 1。若一个样本可以同时属于多个类别（多标签），正确的做法是对每个类别独立使用 sigmoid，而不是 softmax。

补充说明：softmax 在实现时必须先减去最大值。若不减，当某个 z_i 达到几百时，e^{z_i} 会直接溢出为 inf，整个向量变成 nan。减去最大值后，指数项的最大值为 1，其余各项均在 0 到 1 之间，数值上安全，而结果完全不变——因为分子分母同乘了一个常数：

$$\frac{e^{z_i - m}}{\sum_j e^{z_j - m}} = \frac{e^{-m} e^{z_i}}{e^{-m} \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

几乎所有深度学习框架的 softmax 实现都内置了这一步，自己手写时务必补上。

七、隐藏单元与 ReLU

7.1 ReLU 的性质

前面介绍的三种传递函数是输出单元的常见选择，对大部分机器学习模型是通用的。而在深层前馈网络的设计中，一个独有的问题就是隐藏单元的设计，也就是隐藏神经元的传递函数如何选择。但这个问题目前尚不存在放之四海而皆准的通用准则，关于这个环节的有限知识全都集中在一类常用的激活函数上，也就是**整流线性单元**（Rectified Linear Unit，ReLU）。

整流线性单元是隐藏单元理想设计的万金油，当你不知道选择何种激活函数时，用它准保没错。它的数学表达式是：

$$\mathrm{ReLU}(x) = \max(0, x)$$

整流线性单元在一半定义域上取值为 0，在另一半定义域上则处于激活状态。虽然在 x=0 这个不可导的折点上不满足可微的条件，但在激活的区域上，整流线性单元的一阶导数是常数 1，二阶导数则恒等于 0。相比于引入二阶效应的其他激活函数，整流线性单元的梯度方向在学习中具有更大的用处。

补充说明：ReLU 治好了 Sigmoid 的一个老毛病，却带来了一个新毛病。老毛病是**梯度消失**：sigmoid 的导数最大只有 0.25，多层连乘之后梯度迅速衰减到零，深网络根本训不动。ReLU 在正半轴的导数恒为 1，梯度可以一路传回去。新毛病是**神经元死亡**：如果某个神经元的权重在更新后使得它对所有训练样本的输入都为负，那么它的输出恒为 0、梯度恒为 0，参数再也无法更新，这个神经元就永久失效了。学习率设得过大时，这种死亡可能成批发生。Leaky ReLU 与 ELU 正是为此设计的。

7.2 变体与选择

整流线性单元的一个缺点是它可能会“杀死”特定的隐藏神经元，因为一旦取值为 0，这个函数就不会重新激活。解决这个问题的方法是对原始函数做一些微小的修正，得到的变体包括渗漏整流单元（Leaky ReLU）和指数整流单元（Exponential ReLU），它们的共同点是将原来的 0 输出替换为一个较小的数值，以避免神经元的失效。

除了整流线性单元之外，对数几率函数和双曲正切函数等其他函数也可以用于隐藏神经元中，但并不常见。相比于这些函数，整流线性单元的优势在于它的线性特性，这一性质使得模型更加容易优化，对黑箱中的隐藏神经元而言尤其如此。可如果要针对执行特定任务的深度前馈网络选择最优化的隐藏单元，就是个依赖经验的过程。想要预测哪种隐藏单元的性能最佳完全依赖于直觉，至于直觉靠不靠谱，就得通过验证数据集来检验训练效果。

一个实用的顺序是：先试 ReLU，训练不稳定就换 Leaky ReLU 或 ELU，需要输出有界再考虑 tanh。不要在激活函数上反复折腾，架构与数据质量的影响通常大得多。

补充说明：深度网络的训练难题很少能靠换激活函数根治，更常用的是**批归一化**。它在每一层之后对激活值按小批量统计量做标准化再缩放平移，使各层输入的分布保持稳定。这样做的效果有两个：一是允许使用更大的学习率，二是让损失曲面对参数变化不那么敏感，从而缓解梯度消失与初始化敏感的问题。批归一化的代价是训练与推理的行为不一致（推理时要用滑动平均的统计量），这也是它在小批量或序列任务上不如层归一化好用的原因。

四个常见误区

- **"万能逼近定理保证网络能用"**——它只说明存在性，不保证梯度下降能收敛到那组权重，也不保证泛化。
- **"ReLU 随便用就行"**——学习率过大时神经元可能永久失活（死亡 ReLU），Leaky ReLU、ELU 等变体正是为此设计。
- **"损失函数可以随便选"**——损失函数对应着对输出分布的假设。用平方误差做分类，等价于假设高斯噪声，梯度会被错误建模拖慢。
- **"网络越深越强"**——没有残差、归一化与合理初始化，深层网络的训练误差反而更高。

八、要点回顾

1. 深度前馈网络是隐藏层数更多的多层感知器，不使用关于数据结构的先验信息，信息单向流动。
2. 设计的五个核心问题是架构、损失函数、输出单元、隐藏单元与训练误差处理。
3. 链式架构中，层的数目与每层神经元数目是主要变量；全连接参数最多，稀疏连接可大幅减少参数。
4. 通用逼近定理说明单隐藏层网络可以任意精度逼近任意连续函数，但它是存在性定理，不给构造方法。
5. 单隐藏层所需的神经元数目可能大到无法实现，深度架构通过复用降低了表示同一函数所需的参数。
6. 深度架构的有效性依赖于"复杂函数可分解为简单函数的层次组合"这一假设。
7. 回归用均方误差、分类用交叉熵，两者都是最大似然估计在特定分布假设下的具体形式。
8. 最小化交叉熵等价于最小化 KL 散度，因为真实分布的熵与参数无关。
9. 输出单元以线性变换、sigmoid 与 softmax 最为常见；softmax 用于互斥多分类，多标签应用 sigmoid。
10. ReLU 因梯度不衰减而成为默认选择，代价是可能出现神经元死亡，Leaky ReLU 与 ELU 是其变体。
11. 隐藏单元的选择缺乏通用理论，实践中依靠验证集比较；架构与数据的影响通常大于激活函数的差异。

九、名词汇总

名称	英文全称	英文缩写	简短介绍
深度前馈网络	Deep Feedforward Network	DFF	多层、单向传播、无结构先验的神经网络
多层感知器	Multilayer Perceptron	MLP	由输入层、隐藏层、输出层组成的全连接前馈网络
链式架构	Chain Architecture	—	每层输出仅作为下一层输入的串行结构

名称	英文全称	英文缩写	简短介绍
全连接层	Fully Connected Layer	FC	相邻层神经元两两相连的层
稀疏连接	Sparse Connectivity	—	只保留部分连接以减少参数量的结构
通用逼近定理	Universal Approximation Theorem	UAT	单隐藏层网络可任意精度逼近连续函数的存在性结论
表达效率	Representation Efficiency	—	相同函数在不同深度下所需参数的相对多少
均方误差	Mean Squared Error	MSE	预测值与目标值之差的平方的均值
交叉熵	Cross-Entropy	CE	衡量两个概率分布差异的指标，等于真实分布熵与 KL 散度之和
负对数似然	Negative Log-Likelihood	NLL	对数似然取负，分类任务中最常用的损失形式
KL 散度	Kullback-Leibler Divergence	KL	衡量一个分布相对另一个分布的额外信息量
最大似然估计	Maximum Likelihood Estimation	MLE	选择使观测数据概率最大的参数的方法
对数几率函数	Logistic Function	—	把实数压到 0 到 1 之间的 S 形函数，即 sigmoid
柔性最大值函数	Softmax Function	—	把实数向量转为和为 1 的概率分布的归一化指数函数
整流线性单元	Rectified Linear Unit	ReLU	取 max(0,x) 的激活函数，正区间梯度恒为 1
渗漏整流单元	Leaky ReLU	—	负区间给一个小斜率以避免神经元死亡的 ReLU 变体
指数整流单元	Exponential Linear Unit	ELU	负区间用指数函数平滑过渡的 ReLU 变体
双曲正切函数	Hyperbolic Tangent	tanh	输出范围在 −1 到 1 之间的 S 形激活函数
神经元死亡	Dead Neuron	—	因输出恒为 0 且梯度为 0 而永久失效的 ReLU 单元
批归一化	Batch Normalization	BN	按小批量统计量标准化各层激活值的技术
反向传播	Backpropagation	BP	利用链式法则从输出层向输入层逐层计算梯度的算法
随机梯度下降	Stochastic Gradient Descent	SGD	用小批量样本估计梯度并迭代更新参数的优化方法